PyGameMaker — Tutorial 6: Maze — Teacher Guide

Companion for the in-app tutorial (**Help > Tutorials > Maze: Navigate to the Exit**, 5 pages: 4 core + 1 optional bonus) and the matching student handout and worksheet. Students should have finished Tutorial 2 (events, actions, collisions, score).

Overview

Students build a maze game in three phases: a player and walls, coins and an exit, and a score display. New ideas: **smooth movement with held keys**, **facing direction** (a different player sprite per direction), **solid walls**, **collectibles**, an **exit that restarts the room**, and **level design**. An optional 5th page then reuses facing direction to turn the finished maze into a first-person 2.5D game (see "If Time Allows" below) — not part of the core lesson or its timing.

> The maze layout is the creative part. Ask students to draw the maze on grid paper first (the tutorial has a 20×15 example) and check it can be solved before building it.

Suggested Timing (45-60 minutes)

The tutorial says 20-25 minutes for a confident student; building the maze takes the longest.

Segment	Time	What happens
Recap and predict	5 min	Show a finished maze; ask what objects it needs
Phase 1: player and maze	15-20 min	Pages 1-2; most time is spent placing walls
Phase 2: coins and exit	10-15 min	Page 3
Phase 3: score display	5-10 min	Page 4
Swap and play	5 min	Students play each other's mazes
Worksheet / exit ticket	5 min	Worksheet parts A-C

For a 45-minute slot, give students the wall layout on paper, or import the sample sprites, to save time.

Step-by-Step Walkthrough and Common Issues

Phase 1: player and maze

- Five events on the player: four held arrows plus **No key** with *Stop movement*. Without **No key** the player never stops.
- **Facing direction** (4 new sprites + a *Set Sprite* block added to each of the same 4 arrow-key events): purely cosmetic for the core game, but it is the exact pattern the optional bonus page reuses for the raycast camera's facing angle later — worth framing that way even if your class never does the bonus page.
- obj_wall **must be Solid** and the player needs **Collision with obj_wall** with *Stop movement*. Both are needed.

- **Place everything on the grid.** Turn on **Snap to Grid** 32×32. The player moves 4 pixels per step, so if the player or a wall is even a few pixels off the grid the player jams at the entrance of one-tile corridors (a player placed 8 pixels off cannot travel down a corridor at all). "Stuck in corridors" almost always means an off-grid placement; delete and place again with snapping.
- Every corridor must be at least one tile wide, and the maze must be solvable. The tutorial's example maze is 20×15 and solvable; the reference project uses it.

Phase 2: coins and exit

- Coins and exit are **not Solid**; they only react to the collision with the player. The coin uses *Destroy this instance* (the coin). Using *Destroy other instance* here would remove the player.
- *Show message* pauses the game until the student clicks OK; students sometimes think it froze.
- The exit uses *Restart room*: the coins come back. For several levels, use **Go to next room** instead.
- Put the exit far from the start (in the reference maze it is the farthest reachable cell).

Phase 3: score display

- The controller must be placed in the room. Its **Create** event sets the score to 0; because the exit restarts the room, the score resets automatically.
- The score is drawn in white at the top left, over the border wall row, and stays readable.

> **Reference projects.** Download the checkpoint projects from the wiki (06_maze_checkpoints.zip): one project for the end of each phase, with the tutorial's example maze plus five coins and an exit. Give a stuck student the previous phase.

Vocabulary Introduced

Term	What it means here
Held key	An event that is active for as long as the key is down
Facing direction	Which way a character is pointing; drives which sprite shows (and later, the raycast camera's angle)
Solid	Objects that block others (walls)
Collectible	An object that is destroyed when the player touches it
Restart room	Start the level again from the beginning
Snap to Grid	Editor option that aligns objects to a grid

Discussion Questions

- Why do all objects need to be on the grid in this game?
- What makes a maze unfair? (Unreachable coins, no path to the exit.)
- How does the score get back to 0 after winning?
- What would you add to make the game harder?

Differentiation

- **Support:** give the Phase 1 checkpoint and let students build their own maze on top; import the sample sprites.
- **Extension:** more levels with **Go to next room**; a timer; patrolling enemies; keys and doors.

If Time Allows

Page 5 is an **optional bonus**, not part of the core lesson or its timing above: it turns the exact same finished maze into a first-person 2.5D game (Doom/Wolfenstein style), reusing facing direction for the camera instead of a sprite. Five new actions total — one *Enable Raycast View* in the player's **Create** event, and one *Set Facing Angle* added to each of the same 4 arrow-key events (0/180/90/270 for right/left/up/down). The walls need no changes (the *Collision with obj_wall* event from Phase 1 already makes them solid to the player in first person too), and they are textured automatically with each wall's own spr_wall sprite — no wall texture has to be set. The **2.5D Raycast View** extension must be checked in **File > Project Settings... > Extensions** first if the *3D View* category is not already showing in the toolbox. Good for students who finish early; skip it entirely for a 45-60 minute slot.

Worksheet Answer Key

Part A: 1-D, 2-B, 3-A, 4-C.

Part B: 1. obj_player. 2. obj_coin. 3. obj_exit. 4. obj_game_controller. 5. obj_wall (because it is Solid).

Part C:

1. 32 ÷ 4 = 8 steps.
2. The room starts again, so all the coins come back, and the score goes back to 0 because the controller's Create event runs again and sets it.
3. Whether the player and the walls were placed on the grid (Snap to Grid 32×32).

Parts D and E: completion and reflection.

Rubric: Maze

Level	What the game shows
4 - Complete	Player moves and stops correctly; walls block; coins give points and disappear; exit shows a message and restarts; score is displayed; the maze is solvable with every coin reachable
3 - Working	Movement, walls, coins and exit work; the score display is missing, or one small level-design flaw
2 - Partly there	Player moves and walls block, but coins or the exit do not work
1 - Started	Player and walls exist; the player walks through walls or cannot move

End-of-Session Checklist

- [ ] Every student has a saved project (Ctrl+S)
- [ ] Every student reached at least Phase 2 (a coin can be collected)
- [ ] Note who needs the checkpoint project next time